

Stream-uri

Introducere

Începând cu Java 8, limbajul Java a introdus API-ul Streams, una dintre cele mai importante îmbunătățiri ale platformei. Streamurile permit procesarea colecțiilor de date într-un mod:

- mai clar,
- mai concis,
- mai expresiv,
- mai eficient,
- apropiat de paradigma funcțională.

Prin intermediul streamurilor putem realiza operații precum:

- filtrare,
- sortare,
- transformare,
- agregare,
- căutare,
- grupare,
- procesare paralelă,

fără a scrie bucle complexe.

Un Stream reprezintă o secvență de elemente asupra căreia se pot efectua operații pentru procesarea datelor.

Important:

- streamul NU stochează date;
- streamul doar procesează date provenite dintr-o sursă;
- sursa poate fi:
 - colecție (List, Set)

- array
- fișier
- generator de valori
- rezultate din baze de date etc.

Exemplu:

```
//clasic
List<Integer> numere = Arrays.asList(1, 2, 3, 4, 5);

List<Integer> pare = new ArrayList<>();

for(Integer n : numere) {
    if(n % 2 == 0) {
        pare.add(n);
    }
}

System.out.println(pare);

//cu stream
List<Integer> numere = Arrays.asList(1, 2, 3, 4, 5);

List<Integer> pare = numere.stream()
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toList());

System.out.println(pare);
```

Codul cu streamuri:

- este mai scurt;
- este mai lizibil;
- exprimă direct intenția;
- evită gestionarea manuală a colecțiilor intermediare.

Caracteristici

Streamurile au mai multe caracteristici importante.

Nu stocheaza date

Un stream nu conține efectiv elementele.

El doar „curge” peste datele existente.

Exemplu:

```
List<String> lista = Arrays.asList("Ana", "Maria", "Ion");
```

```
Stream<String> stream = lista.stream();
```

Datele sunt în lista, nu în stream.

Nu modifică sursa datelor

Operațiile pe stream nu modifică lista originală.

Exemplu:

```
List<Integer> numere = Arrays.asList(1, 2, 3, 4);
```

```
List<Integer> rezultat = numere.stream()  
    .filter(n -> n % 2 == 0)  
    .collect(Collectors.toList());
```

```
System.out.println(numere);
```

Lista inițială rămâne:

```
[1, 2, 3, 4]
```

Sunt consumabile o singură dată

După utilizare, streamul nu mai poate fi reutilizat.

Exemplu:

```
Stream<String> stream = Stream.of("A", "B", "C");  
  
stream.forEach(System.out::println);  
  
stream.forEach(System.out::println); // EROARE
```

Rezultă:

`java.lang.IllegalStateException`

Suportă procesare funcțională

Streamurile utilizează:

- expresii lambda;
- interfețe funcționale;
- programare declarativă.

Exemplu:

```
numere.stream()
    .map(n -> n * 2)
    .forEach(System.out::println);
```

Pot fi paralele

Java permite procesarea paralelă foarte ușor.

Exemplu:

```
numere.parallelStream()
    .forEach(System.out::println);
```

Avantaj:

- folosirea mai multor nuclee ale procesorului;
- performanță mai bună pentru volume mari de date.

Evaluare leneșă (Lazy Evaluation)

Operațiile intermediare nu se execută imediat.

Execuția începe doar când apare o operație terminală.

Exemplu:

```
Stream<Integer> stream = numere.stream()
    .filter(n -> {
        System.out.println(n);
        return n % 2 == 0;
    });
```

Până nu apare o operație terminală, nimic nu se execută.

Pot fi infinite

Streamurile pot genera secvențe infinite.

Exemplu:

```
Stream<Integer> infinit = Stream.iterate(0, n -> n + 1);
```

De aceea este necesară limitarea:

```
infinit.limit(10)  
    .forEach(System.out::println);
```

Etapele utilizării unui stream

Utilizarea unui stream implică trei etape principale:

1. Crearea streamului
2. Operații intermediare
3. Operații terminale (de închidere)

Crearea streamului

Un stream poate fi creat din mai multe surse.

Din colecții

Cea mai frecventă metodă.

```
List<String> nume = Arrays.asList("Ana", "Ion", "Maria");  
  
Stream<String> stream = nume.stream();
```

Din array-uri

```
int[] valori = {1, 2, 3, 4};  
  
IntStream stream = Arrays.stream(valori);
```

Cu Stream.of()

```
Stream<String> stream = Stream.of("Java", "Python", "C++");
```

Stream infinit cu iterate()

```
Stream<Integer> stream = Stream.iterate(1, n -> n + 1);
```

Stream infinit cu generate()

```
Stream<Double> random = Stream.generate(Math::random);
```

Din fișiere

```
try(Stream<String> linii = Files.lines(Paths.get("fisier.txt"))) {  
    linii.forEach(System.out::println);  
}
```

Streamuri primitive

Java oferă streamuri speciale:

- IntStream
- LongStream
- DoubleStream

Exemplu:

```
IntStream.range(1, 5)  
    .forEach(System.out::println);
```

Operații intermediare

Operațiile intermediare:

- transformă streamul;
- returnează un nou stream;
- NU produc rezultat final;
- sunt lazy.

filter()

Filtrează elementele cu ajutorul unui Predicate.

```
numere.stream()
    .filter(n -> n % 2 == 0)
    .forEach(System.out::println);
```

Rezultat:

2
4

map()

Transformă elementele.

Exemplu:

```
nume.stream()
    .map(String::toUpperCase)
    .forEach(System.out::println);
```

Rezultat:

ANA
ION
MARIA

sorted()

Sortează elementele cu ajutorul unui Comparator.

```
numere.stream()
    .sorted()
    .forEach(System.out::println);
numere.stream()
    .sorted((a, b) -> b - a)
    .forEach(System.out::println);
```

4.4 distinct()

Elimină duplicatele.

```
Stream.of(1, 2, 2, 3, 3, 4)
    .distinct()
    .forEach(System.out::println);
```

limit()

Limitează numărul de elemente.

```
Stream.iterate(1, n -> n + 1)
    .limit(5)
    .forEach(System.out::println);
```

skip()

Sare peste elemente.

```
numere.stream()
    .skip(2)
    .forEach(System.out::println);
```

peek()

Permite inspectarea elementelor.

Folosit frecvent pentru debugging.

```
numere.stream()
    .peek(System.out::println)
    .map(n -> n * 2)
    .forEach(System.out::println);
```

flatMap()

Transformă și aplatizează structuri complexe.

Exemplu:

```
List<List<String>> liste = Arrays.asList(
    Arrays.asList("A", "B"),
    Arrays.asList("C", "D")
);
```



```
liste.stream()  
    .flatMap(List::stream)  
    .forEach(System.out::println);
```

Rezultat:

A
B
C
D

Operații terminale (de închidere)

Operațiile terminale:

- finalizează streamul;
- produc rezultatul final;
- declanșează execuția.

forEach()

Parcurge elementele.

```
nume.stream()  
    .forEach(System.out::println);
```

collect()

Colectează rezultatele.

```
List<String> lista = nume.stream()  
    .map(String::toUpperCase)  
    .collect(Collectors.toList());
```

count()

Numără elementele.

```
long nr = numere.stream()
    .filter(n -> n > 5)
    .count();
```

reduce()

Reduce elementele la o singură valoare.

Exemplu: sumă.

```
int suma = numere.stream()
    .reduce(0, (a, b) -> a + b);
```

findFirst()

Returnează primul element.

```
Optional<Integer> primul =
    numere.stream().findFirst();
```

findAny()

Returnează un element arbitrar.

```
Optional<Integer> element =
    numere.stream().findAny();
```

anyMatch()

Verifică dacă există elemente care respectă o condiție.

```
boolean exista =
    numere.stream()
        .anyMatch(n -> n > 10);
```

allMatch()

```
boolean toate =  
    numere.stream()  
        .allMatch(n -> n > 0);
```

noneMatch()

```
boolean niciunul =  
    numere.stream()  
        .noneMatch(n -> n < 0);
```

min() și max()

```
Optional<Integer> minim =  
    numere.stream().min(Integer::compare);  
  
Optional<Integer> maxim =  
    numere.stream().max(Integer::compare);
```

Fluxul complet al unui stream

Exemplu complet:

```
List<String> nume = Arrays.asList(  
    "Ana",  
    "Ion",  
    "Maria",  
    "Alex"  
);  
  
List<String> rezultat =  
    nume.stream()  
        .filter(n -> n.length() > 3)  
        .map(String::toUpperCase)  
        .sorted()  
        .collect(Collectors.toList());
```

```
System.out.println(rezultat);
```

Avantajele streamurilor

Cod mai scurt - Reduc cantitatea de cod.

Lizibilitate crescută - Descriu „ce” se face, nu „cum”.

Programare declarativă - Codul devine apropiat de limbaj natural.

Procesare paralelă simplă - `parallelStream()`

Mai puține erori

Se evită:

- indexări greșite,
- manipulări manuale,
- colecții intermediare inutile.

Dezavantajele streamurilor

Debugging mai dificil - Lanțurile lungi pot fi greu de urmărit.

Consum suplimentar de memorie - Uneori apar obiecte intermediare.

Nu sunt ideale pentru orice situație - Pentru logici foarte complexe, buclele clasice pot fi mai clare.

Diferența dintre Collection și Stream

Collection	Stream
Stochează date	Procesează date
Se poate reutiliza	Se consumă o singură dată
Operații imperative	Operații declarative
Evaluare imediată	Evaluare lazy

Exercitii

1. Creeaza o clasa Student cu campurile:

- Name
- Age
- Grade
- University

- a) Creeaza o lista cu 5-6 studenți.
- b) Afiseaza numele tuturor studentilor cu media peste 5
- c) Creați o listă care conține numele studenților transformate cu litere mari și sortate alfabetic.
- d) Determinați numărul studenților de la facultatea „Informatica”.
- e) Verificați dacă există cel puțin un student minor.
- f) Determinați studentul cu media cea mai mare.
- g) Calculați media tuturor mediilor.
- h) Afișați primii 3 studenți sortați descrescător după medie.

2. Creeaza o clasa Order cu campurile:

- Id
 - Product
 - Price
 - Quantity
 - isDelivered
- a) adauga 5-6 comenzi
 - b) Afișați toate comenzile livrate.
 - c) Creați o listă cu numele produselor fără duplicate.
 - d) Calculați valoarea totală a tuturor comenzilor.
 - e) Afișați produsele sortate după preț descrescător.
 - f) Verificați dacă toate comenzile au cantitate pozitivă.
 - g) Determinați cea mai ieftină comandă.
 - h) Afișați primele 2 produse cele mai scumpe.
 - i) Săriți peste primele 3 comenzi și afișați restul.

3. Se considera lista de cuvinte:

"java",

"stream",
"lambda",
"programare",
"java",
"colectii",
"functional",
"stream"

- a) Afișați toate cuvintele care au lungime mai mare de 5.
- b) Transformați toate cuvintele în litere mari.
- c) Eliminați duplicatele și sortați alfabetic.
- d) Calculați numărul total de caractere din listă.
- e) Verificați dacă există cuvinte care conțin litera „z”.
- f) Determinați cel mai lung cuvânt.
- g) Concatenați toate cuvintele într-un singur șir separat prin virgulă.
- h) Afișați procesul de transformare utilizând peek().

4. Creați 2 clase: Employee și Department cu campurile:

- Name
- Salary

- Name
- EmployeeList

- a) Adăugați 10 angajați și 3 departamente.
- b) Afișați numele tuturor angajaților din toate departamentele.
- c) Determinați salariul total al tuturor angajaților.
- d) Afișați angajații cu salariu peste 5000.
- e) Sortați toți angajații după salariu descrescător.
- f) Determinați angajatul cu salariul maxim.
- g) Verificați dacă există departamente fără angajați.
- h) Afișați primele 5 salarii distincte în ordine descrescătoare.
- i) Utilizați peek() pentru a afișa fiecare angajat înainte și după transformare.